

Join

Join

Come collegare tabelle diverse con *INNER JOIN*, *LEFT JOIN*, *RIGHT JOIN* e *FULL JOIN*.

Mettere insieme dati che vivono in tabelle diverse

In un database relazionale, le informazioni non stanno sempre tutte nella stessa tabella.

Per esempio, potresti avere una tabella `clienti` e una tabella `ordini`. La prima contiene le persone. La seconda contiene gli acquisti.

Da sole sono utili, ma insieme raccontano una storia più completa: **chi ha comprato cosa**.

Le `JOIN` servono proprio a questo: collegare tabelle diverse mentre leggi i dati.

Un primo esempio

Immagina queste due tabelle.

clienti

id	nome
1	Luca
2	Marta
3	Sara

ordini

id	cliente_id	totale
10	1	49.90
11	2	18.50

Nella tabella `ordini`, la colonna `cliente_id` dice a quale cliente appartiene ogni ordine.

Per mostrare il nome del cliente insieme al totale dell'ordine, scrivi:

```
SELECT clienti.nome, ordini.totale
FROM clienti
INNER JOIN ordini ON clienti.id = ordini.cliente_id;
```

Il risultato sarà:

nome	totale
Luca	49.90
Marta	18.50

Sara non appare perché non ha ordini collegati.

Cosa fa **INNER JOIN**

INNER JOIN tiene solo le righe che trovano una corrispondenza in entrambe le tabelle.

Nel nostro esempio:

- Luca ha un ordine, quindi appare.
- Marta ha un ordine, quindi appare.
- Sara non ha ordini, quindi non appare.

È come fare l'appello usando due liste e tenere solo i nomi che riesci ad abbinare.

La parte più importante: **ON**

Questa riga è il collegamento:

```
ON clienti.id = ordini.cliente_id
```

Significa:

"Abbina un cliente a un ordine quando l'id del cliente è uguale al `cliente_id` salvato nell'ordine."

Se sbagli questa condizione, la join collega righe sbagliate. Per questo conviene leggerla sempre con calma.

Perché scriviamo `clienti.nome`

Quando usi più tabelle, possono esistere colonne con lo stesso nome.

Per esempio, sia `clienti` sia `ordini` hanno una colonna `id`. Scrivere `clienti.id` o `ordini.id` evita ambiguità.

È come usare nome e cognome quando in una stanza ci sono due persone chiamate Luca.

LEFT JOIN : tenere anche chi non ha corrispondenze

A volte vuoi vedere tutti i clienti, anche quelli che non hanno ancora fatto ordini.

In quel caso usi `LEFT JOIN` :

```
SELECT clienti.nome, ordini.totale
FROM clienti
LEFT JOIN ordini ON clienti.id = ordini.cliente_id;
```

Il risultato diventa:

nome	totale
Luca	49.90
Marta	18.50
Sara	NULL

`NULL` significa che manca un valore. Qui vuol dire: "per Sara non esiste un ordine collegato".

Quale join scegliere?

All'inizio concentrati su queste due:

- `INNER JOIN` quando vuoi solo le righe che si abbinano.

- `LEFT JOIN` quando vuoi tenere tutte le righe della tabella di sinistra.

Esistono anche `RIGHT JOIN` e `FULL JOIN`, ma puoi incontrarle più avanti. Nelle query quotidiane, `INNER JOIN` e `LEFT JOIN` coprono già tantissimi casi.

Riepilogo rapido

- Una `JOIN` legge insieme dati di tabelle diverse.
- `INNER JOIN` mostra solo le corrispondenze.
- `LEFT JOIN` tiene anche le righe senza corrispondenza a destra.
- `ON` spiega come collegare le tabelle.
- Scrivere `tabella.colonna` evita confusione quando più tabelle hanno colonne simili.